

Robotic Car Painting Cell Simulation

Aditya Pachauri

1. Objective, software, and cell layout

The task was to design a simple simulated car-painting cell that demonstrates a car body, one industrial robot, a paint tool, robot motion, paths on different exterior surfaces, and basic collision checking. The solution uses **ROS 2 Jazzy**, **Gazebo Harmonic**, **MoveIt 2**, and **RViz 2** on Ubuntu 24.04. Gazebo provides the physics and joint simulation; MoveIt handles inverse kinematics, free-space planning, Cartesian paths, and collision queries; RViz displays the robot, planning scene, paths, and spray state.

Component	Implemented specification
Vehicle	OpenRobotics Hatchback v3 scaled to exactly $4.8 \times 1.9 \times 1.6$ m
Robot	Universal Robots UR20; six revolute joints and 1.75 m nominal reach
Placement	Car at bay centre (0, -1.5, 0) m; robot base at (0.4, -2.9, 0.9) m
Tool	200 mm spray nozzle fixed to tool0; <code>spray_nozzle</code> is the planning TCP
Painting	Three serpentine rows each on hood, roof, and near side panel
Target standoff	0.25 m from the sampled mesh surface

The assignment drawing is a general layout reference rather than a mandatory machine arrangement. This implementation follows its main dimensions and topology: a **12 × 8 m overall cell**, an **8 × 5 m central working bay**, two **2 × 5 m side bays**, and a **3 m deep upper service row**. Both the outer and inner side walls have aligned **2.2 m openings**, giving the 1.9 m wide car a continuous entry-to-exit route with 0.15 m nominal side clearance. Internal walls form three service rooms; two openings connect the service row. A control cabinet, paint supply, and extraction unit occupy those rooms. The car is centred in the clear painting bay.

The layout is defined once in `config/cell.yaml`. The launch file converts those boxes into Gazebo SDF models, while the demo inserts the same collision-enabled boxes into the MoveIt planning scene. This avoids a visual/planning mismatch: a wall visible in Gazebo is also an obstacle used during planning.

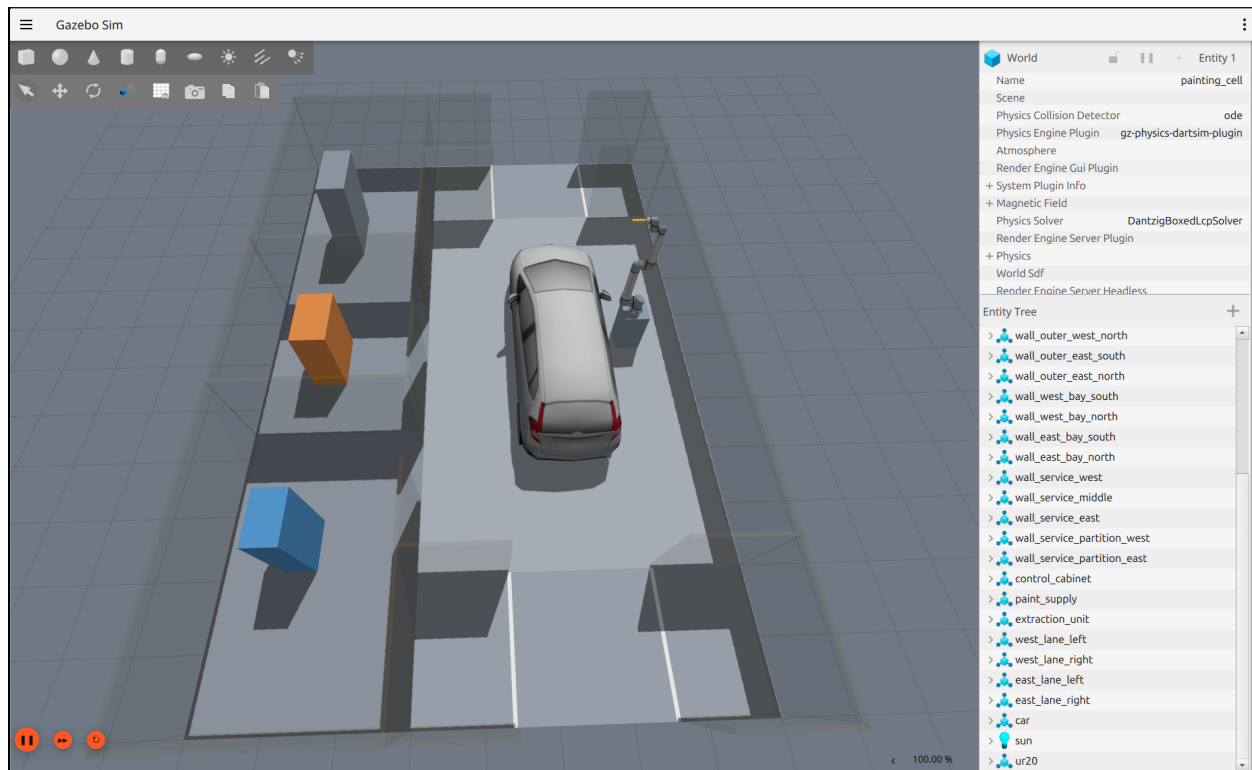


Figure 1: Live Gazebo Harmonic simulation environment showing the UR20 robot, car, pedestal, and booth layout.

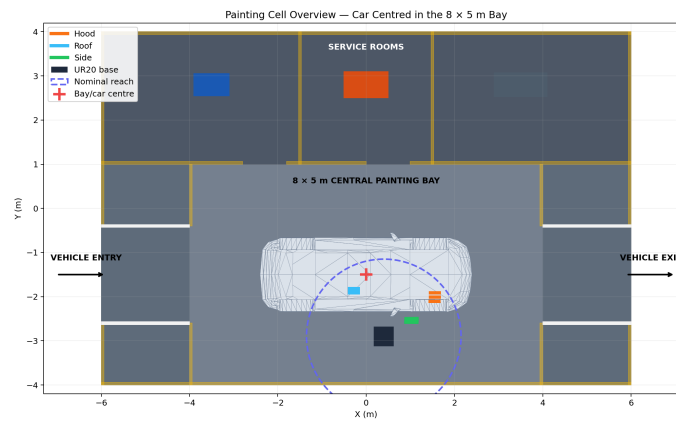


Figure 2: Reference-style cell layout diagram showing car, robot base, nominal reach, and paint paths.

2. Models, URDF/Xacro, frames, and ROS 2 control

Car preparation

The downloaded Hatchback v3 OBJ was rotated into the project convention, centred in X and Y, grounded at $Z = 0$, and scaled independently on each axis. `prepare_car.py` checks the resulting bounding box. The submitted collision and visual mesh has minimum coordinates $(-2.4, -0.95, 0.0)$ m and dimensions $(4.8, 1.9, 1.6)$ m. Gazebo loads it through `model.sdf`; MoveIt loads the same triangles as a `CollisionObject`, so path generation, display, and collision checking refer to one car surface.

Robot and painting tool

The manufacturer-maintained `ur_description` package supplies UR20 dimensions, joint limits, kinematic calibration, inertial data, collision STL files, and visual DAE files. Those upstream robot files were not remodelled. `cell.urdf.xacro` acts as a wrapper: it instantiates the UR20 macro, fixes the robot to world at the configured pedestal pose, adds the tool, and enables Gazebo `ros2_control`.

The custom `spray_nozzle` link is a 0.20 m long, 0.025 m radius cylinder with visual, collision, mass, and inertia elements. A fixed `spray_tool_joint` attaches it 0.20 m along the flange Z axis. Its cylinder extends behind the `spray_nozzle` frame, leaving that frame at the tool tip. Therefore the commanded TCP is the nozzle tip, and the full nozzle body still participates in collision checking.

The main coordinate chain is:

```
world -> base_link -> shoulder -> upper arm -> forearm
      -> wrist 1 -> wrist 2 -> wrist 3 -> tool0
      -> spray_nozzle (TCP)
```

The SRDF defines `ur_manipulator` from `base_link` to `spray_nozzle`. It disables only adjacent or manufacturer-designated never-colliding robot link pairs; robot-to-car, robot-to-wall, and nozzle-to-car collisions remain enabled. The Xacro also declares a GazeboSystem hardware interface. `joint_state_broadcaster` publishes feedback for all six joints, and `joint_trajectory_controller` accepts `FollowJointTrajectory` goals. `robot_state_publisher` converts those joint values into TF transforms.

URDF/Xacro describes the moving robot and tool. SDF describes the Gazebo world, car, floor, pedestal, walls, and service equipment. MoveIt uses the URDF/SRDF robot plus planning-scene copies of the SDF obstacles.

MoveIt Planning Scene Geometry Matches the Simulated Cell

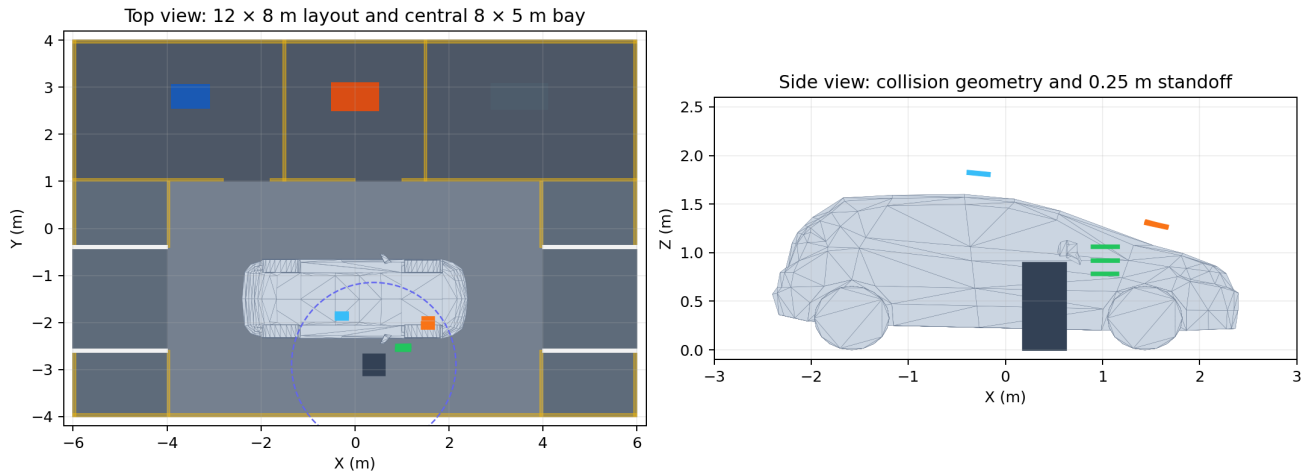


Figure 3: Top and side views of the MoveIt planning scene geometry matching the simulated cell layout.

3. Surface sampling and path delivery to the arm

Painting paths are generated from the submitted car mesh rather than drawn in free space. `cell.yaml` gives small safe regions for the hood, roof, and rear side. Each row contains nine samples and alternate rows reverse direction, producing a serpentine raster with fewer non-painting reposition moves.

For each sample, `geometry.py` casts a ray into the vehicle and uses the Möller-Trumbore ray/triangle intersection algorithm. The nearest positive intersection returns surface point p and triangle normal n_i . Normals across one row are averaged and normalized to n . A row is rejected if any local normal differs from the average by more than 20 degrees, which prevents a path from crossing a sharp feature such as a roof edge.

The nozzle target is constructed as

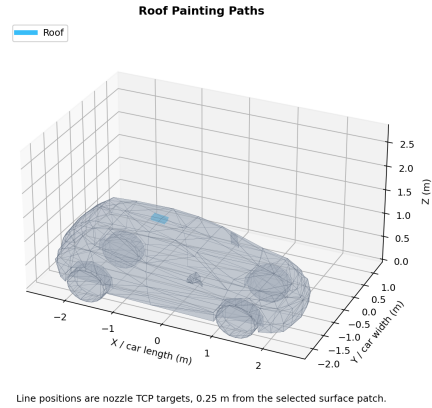
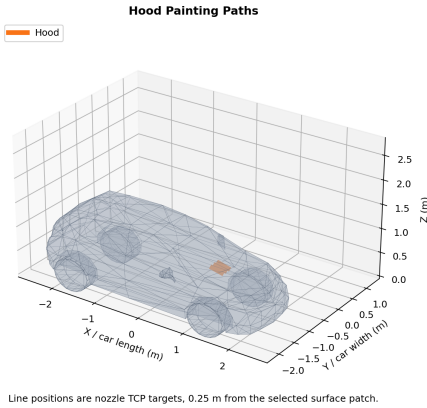
$$p_{TCP} = p + d n, \quad d = 0.25 \text{ m}.$$

The tool local +Z axis is aligned to $-n$, so the nozzle points toward the panel. A stable perpendicular X axis completes the orientation matrix, which is converted to a quaternion. The resulting nine rows cover three distinct surface orientations: horizontal front bodywork, elevated roof, and vertical side bodywork.

MoveIt receives these poses in four stages:

1. `compute_ik` searches for a joint solution at the first TCP pose. Several seeds are tried because a six-axis arm can reach one pose using different elbow/wrist branches.
2. The complete row is tested with `compute_cartesian_path`. A branch is accepted only when the full row is continuous and its completion fraction is at least 0.999.
3. OMPL RRTConnect plans a collision-aware spray-OFF transition from the previous row to the accepted start state.
4. The accepted Cartesian row is retimed toward 0.08 m/s and sent to Gazebo through `FollowJointTrajectory`; spray state is ON only for this row.

The configured point-to-surface construction is exactly 0.250 m at every ray hit. An additional nearest-triangle test samples 441 points along the interpolated TCP rows; the measured TCP-to-mesh clearance is **0.2462 to 0.2502 m**. The small difference comes from curvature between adjacent ray hits. This is suitable for the selected smooth demonstration patches. It is a geometric TCP spacing result, not proof of real paint film thickness or airflow behaviour. MoveIt separately checks the 25 mm radius nozzle body for collision.



4. Link collision avoidance and verified results

The robot avoids the car through geometry checks. The demo first applies 21 world collision objects to MoveIt: the triangle car mesh, floor, pedestal, 15 outer/internal wall sections, and three service units. Collision-aware IK rejects invalid endpoint states. OMPL checks free-space transitions, and the Cartesian service runs with `avoid_collisions=true`.

After planning, the project performs a second explicit validation pass. Every trajectory is interpolated so the maximum change of any joint between checked states is at most 0.04 rad. `GetStateValidity` then tests the complete URDF robot state against self-collision and every planning-scene object. A collision names both bodies and aborts the sequence before controller execution. The initial simulated state is checked in the same way.

A negative test proves that the car collision mesh is active. Collision-aware IK is temporarily disabled to construct a pose 30 mm inside the first panel. `GetStateValidity` detects contact between car and `spray_nozzle`. The state is recorded with `executed: false` and is never sent to the controller.

Verified metric	Final run
Painting strokes completed	9 of 9
Cartesian completion	1.0 for every stroke
Executed segments	18 (9 spray-OFF approaches + 9 spray-ON rows)
Interpolated robot states checked	1,197
Active planning-scene objects	21
Target standoff recorded	0.250000 m min/max within floating-point precision
Dense nearest-mesh TCP clearance	0.2462 to 0.2502 m
Negative collision test	Passed; colliding state not executed
Simulated sequence duration	117.184 s
Maximum final joint error	0.001385 rad
Maximum selected-row normal deviation	8.454 degrees

The values above come from the completed Gazebo/MoveIt run in `results/execution.json`. The result file records every row's Cartesian fraction, collision sample count, TCP endpoints, target standoff bounds, rejected IK branches, segment duration, spray state, and final joint error.

```
$ cat results/execution.json

status:                executed
robot:                 UR20
car_dimensions_m:      [4.8, 1.9, 1.6]
validated painting strokes: 9
Cartesian fractions:
  hood:                hood_1=1.0, hood_2=1.0, hood_3=1.0
  roof:                roof_1=1.0, roof_2=1.0, roof_3=1.0
  side:                side_1=1.0, side_2=1.0, side_3=1.0
executed approach/paint segments: 18
planning-scene collision objects: 21
collision samples checked: 1197
target standoff range_m: 0.250000 .. 0.250000
negative collision test: PASSED; contacts=[['car', 'spray_nozzle']]
negative test executed: False
simulated sequence duration_s: 117.184
maximum endpoint error_rad: 0.001385

All reported values are generated from the completed Gazebo/MoveIt execution.
```

Figure 4: Machine-readable evidence rendered from the final execution report.

5. Code structure, reproduction, results, and limits

The submitted project structure:

File	Purpose
config/cell.yaml	Robot pose, standoff, panel regions, and complete shared layout
urdf/cell.urdf.xacro	UR20 wrapper, nozzle link/joint, and Gazebo control interface
worlds/cell.sdf	World template populated from the YAML layout
launch/cell.launch.py	Starts Gazebo, controllers, MoveIt, RViz, and builds layout boxes
geometry.py	OBJ loading, ray casting, normals, TCP orientation, and clearance calculation
demo.py	Planning scene, IK, OMPL/Cartesian planning, validation, execution, and evidence
painting.srdf	Planning group and limited allowed-collision matrix
controllers.yaml	Six-joint ROS 2 trajectory controller configuration

To reproduce the result, build with `colcon build --symlink-install`, source `scripts/env.zsh` in Zsh (or `scripts/env.sh` in Bash), and start:

```
ros2 launch painting_cell cell.launch.py gui:=true rviz:=true
```

In a second sourced terminal, run:

```
ros2 run painting_cell painting_demo --execute \
  --output results/execution.json --keep-alive
```

All core checks verify exact car dimensions, panel groups, serpentine direction, 250 mm target construction, tool orientation, clearance to the real mesh, reference-layout dimensions, continuous vehicle route, collision boxes, and spray-cone marker geometry.

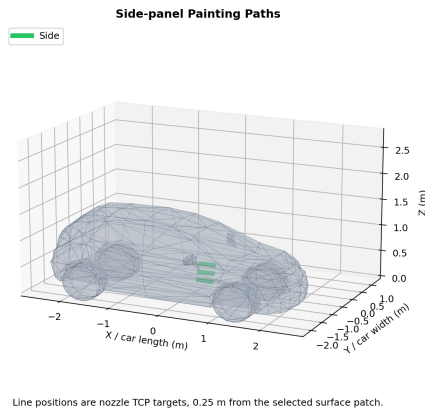


Figure 5: Side-panel paths demonstrate tool orientation and vertical surface coverage.

This remains an educational simulation. It paints selected patches rather than the entire vehicle. It does not model paint droplets, fan overlap, coating thickness, hoses, dynamic vehicle transport, safety PLCs, ventilation, or certified safety distances. A real automotive booth would normally use paint-rated explosion-protected hardware, certified spray and extraction equipment, and often a second robot or linear rail. The UR20 was chosen here for its maintained ROS 2 Jazzy description and practical MoveIt/Gazebo integration.

The final result satisfies the requested simulation scope: the reference layout is visibly modeled, the same objects participate in MoveIt collision checking, paths are derived from the submitted car mesh, the nozzle orientation and distance are quantified, and all nine rows execute successfully in Gazebo.